Veridise

Auditing Report

Hardening Blockchain Security with Formal Methods

FOR

RISC ZERO

RISC Zero M3 zkVM



Veridise Inc.

February 12, 2026

Last updated date: Apr. 07, 2026

► **Prepared For:**

RISC Zero
https://risczero.com/

► **Prepared By:**

Shankara Pailoor
Ajinkya Rajput
Benjamin Sepanski
Bryan Tan

► **Contact Us:**

contact@veridise.com

► **Version History:**

Apr. 07, 2026 V2 - Updated issue statuses and fixed typos
Apr. 03, 2026 V1

© 2026 Veridise Inc. All Rights Reserved.

Contents

Contents iii

1 Executive Summary 1

2 Project Dashboard 4

3 Security Assessment Goals and Scope 5
3.1 Security Assessment Methodology & Scope . . . 5
3.2 Classification of Vulnerabilities . . . 6

4 Trust Model 8
4.1 Operational Assumptions . . . 8

5 Vulnerability Report 9
5.1 Detailed Description of Issues . . . 10
5.1.1 V-R0M3-VUL-001: Underconstrained UnitDivBlock . . . 10
5.1.2 V-R0M3-VUL-002: Underconstrained DivBlock due to division-by-zero . 11
5.1.3 V-R0M3-VUL-003: Underconstrained P2StepBlock . . . 12
5.1.4 V-R0M3-VUL-004: Underconstrained cycleCount in EcallBigIntBlock . . 13
5.1.5 V-R0M3-VUL-005: Consecutive BigInt rows not guaranteed to be of consecutive cycles . . . 15
5.1.6 V-R0M3-VUL-006: JALR target computation does not clear bit 0 before next-PC transition . . . 16
5.1.7 V-R0M3-VUL-007: Missing validation of unaligned, odd-numbered program counter . . . 17
5.1.8 V-R0M3-VUL-008: User mode can decode full instruction partially contained in kernel memory . . . 19
5.1.9 V-R0M3-VUL-009: BigIntBlock::verify does not check valid polyOps with CHECK memOp . . . 21
5.1.10 V-R0M3-VUL-010: AddressVerify::verify isMM value is non-boolean for supervisor mode . . . 23
5.1.11 V-R0M3-VUL-011: DecodeBlock::set uses an inconsistent unaligned-PC predicate . . . 24
5.1.12 V-R0M3-VUL-012: BigIntBlock::applyInner does not apply prevCycle and prevValue . . . 26
5.1.13 V-R0M3-VUL-013: Claim decoding panics on invalid seal data . . . 27
5.1.14 V-R0M3-VUL-014: Minor code quality and maintainability issues . . . 29

6 Fuzz Testing 32
6.1 Methodology . . . 32
6.1.1 Arguzz Overview . . . 32
6.1.2 Adaptation to the RISC Zero M3 Backend . . . 32
6.1.3 Campaign Configuration . . . 33
6.1.4 Coverage and Limitations . . . 33
6.1.5 Results . . . 33

 © 2026 Veridise Inc.

7 Formal Verification **34**
7.1 Background . . . 34
7.2 Methodology . . . 34
7.3 Properties Validated . . . 35
7.3.1 Determinism of Components . . . 35
7.3.2 Other Functional Correctness Properties . . . 36
7.4 Recommendations . . . 38
7.4.1 Determinism Annotations . . . 38
7.4.2 Bounds Post-conditions . . . 39
7.4.3 Argument and Component Invariants . . . 40

A Appendix **41**
A.1 Intended Behavior: Non-Issues of Note . . . 41
A.1.1 V-R0M3-APP-VUL-001: Underconstrained BigInt NOP write . . . 41
A.1.2 V-R0M3-APP-VUL-002: EcallTerminateBlock does not validate halt state 43

Glossary **44**

 © 2026 Veridise Inc.

About Veridise

Veridise is an independent security firm founded by academics and security researchers with deep expertise in formal methods, programming languages, and applied cryptography. The firm focuses on high-assurance security for blockchain and cryptographic systems, combining rigorous theory with practical adversarial analysis.

Veridise provides full-stack blockchain security services spanning smart contracts, zero-knowledge circuits and proving systems, consensus and execution clients, cryptographic libraries, and supporting infrastructure. The team routinely analyzes systems at the boundary between protocol design, implementation, and cryptography, where failures are both subtle and high-impact. To date, Veridise has conducted hundreds of security assessments for protocols securing billions of dollars in TVL.

Dynamic Security Veridise treats security as an ongoing engineering discipline rather than a one-time validation exercise. Effective security requires iterative review, continuous feedback, and close collaboration between auditors and system designers. Throughout the engagement, Veridise analysts communicated regularly with the RISC Zero team using AuditHub, enabling rapid clarification of design intent, prompt discussion of findings, and efficient validation of fixes.

In parallel with manual review, Veridise maintains active research programs focused on automated and formal analysis of blockchain systems. These efforts have produced tools such as *Vanguard* for static analysis and *Picus* for formal verification, which are available through AuditHub. AuditHub integrates these techniques directly into CI/CD workflows, supporting continuous vulnerability detection and regression analysis beyond the scope of a single audit.

Additional details and case studies are available at `https://audithub.dev/case-studies/`.

Veridise Reports Veridise reports are published in the public audit archive linked below. Only reports obtained from this archive or confirmed directly by a representative of Veridise should be considered official Veridise reports.

`https://veridise.com/audits-archive/`

© 2026 Veridise Inc.

1 Executive Summary

From Feb. 12, 2026 to Mar. 27, 2026, RISC Zero engaged Veridise to conduct a security assessment of several major components of their RISC Zero M3 zkVM (abbreviated in this report as R0VM M3). Veridise conducted the assessment over 21 person-weeks, with 3 security analysts reviewing the project over 7 weeks on commit `da8083dc` of the `risc0/risc0` GitHub repository and commit `df6fb9dd` of the `risc0/zirgen` GitHub repository. The review strategy involved a tool-assisted analysis of the program source code performed by Veridise security analysts as well as thorough code review.

Project Summary. The RISC Zero M3 zkVM is the third version of the RISC Zero zkVM, whose zero-knowledge circuit has been adapted to use the M3 arithmetization scheme. The security assessment covered several major components of the zkVM, including: a C++ implementation of the RISC-V instruction set as a zero-knowledge circuit, surrounding C++ code that uses the circuit to implement various subroutines (see below), and logic to compile the circuit to Rust and Zirgen source code that implements a verifier. A prior "V2" version of the circuit implemented in Zirgen has been previously reviewed by Veridise*. Throughout the report, we will use terminology specific to RISC Zero, as described in their developer documentation.

Circuit Features. The main component of the security assessment is the zero-knowledge circuit, which implements the RV32IMC instruction set† along with user and machine privilege modes. Additionally, when in the machine privilege mode, the machine supports several intrinsics through the `ECALL` instruction, including the Poseidon2 hash function, arbitrary-size big integer (BigInt) operations, external read/write, and an operation to terminate a zkVM session.

The machine's address space is divided into two regions: user space and kernel space. The user space memory generally stores data such as the guest program, the program stack, the program heap, etc. and can be accessed in all privilege modes. In contrast, kernel space memory stores internal machine state such as the kernel trap handler program, the register files for each privilege mode, etc. and can only be accessed in the machine privilege mode.

Circuit Implementation. Internally, the circuit is organized as a set of *blocks*, each performing some fixed functionality. For example, the `UnitAddSubBlock` performs a signed 32-bit integer addition operation. Every block has a list of registers or sub-blocks holding data related to that block, with the semantics of the block determined by a set of constraints imposed on the values of those registers/sub-blocks. To enable communication between blocks, each block may indicate *arguments* which are either provided by (pulled) or sent to (pushed) other blocks‡.

A segment in an M3 zkVM computation trace is represented by a sequence of *rows*, where each row corresponds to the execution of exactly one type of block at exactly one point in time. Note that the rows generally do not include any built-in notion of "CPU cycle", nor do they have any meaningful ordering; rather, each block may contain a cycle register, with relationships between different rows (i.e., blocks) enforced through arguments.

* The audit report for the previous version can be found on RISC Zero's security resources at https://github.com/risc0/rz-security/blob/2eb865f9b37bca0d6be3f15de768b630f797c66f/audits/zkVM/veridise_zkVM_20250224.pdf

† The RV32I base instructions with the compression ("C") and multiplication ("M") extensions

‡ Arguments correspond to the *channels* in M3 arithmetization.

© 2026 Veridise Inc.

Supplementary Circuit Code. The circuit can be instantiated under a so-called context to perform a specific task on a single row; the task can be performed on an entire segment by using the context on all of the rows. The contexts which were covered during the security assessment include:

1. `VerifyContext` implements verification logic for a single row, by transforming the constraints and arguments of all blocks into a polynomial. The validity of a witness can be checked by evaluating the polynomial at a challenge point[§].
2. `DataWitgenContext` and `AccumWitgenContext` generates witness information related to a single row, such as register values and argument information.
3. `RecordingContext` generates an implementation of a verifier for all of the blocks in the M3 circuit as zirgen code, mainly used for code generation purposes.

The other components covered in the security assessment involve the verification logic, which is implemented using a mixture of C++ and Rust. The C++ code uses the `RecordingContext` to generate Zirgen modules which encode the circuit's constraints as the evaluation of a polynomial at a challenge point, to be used in a verifier implementation. These Zirgen modules can be converted into Rust source code implementing the polynomial evlauation, or to a RISC Zero zkVM recursion program (`.zkr` file).

Code Assessment. The RISC Zero developers provided the source code contained in the `risc0/risc0` and `risc0/zirgen` GitHub repositories for the code review. The source code corresponding to the circuit appears to be a port of the RISC Zero zkVM V2 circuit code (originally written in Zirgen) to C++, but with several new features, additional bug fixes, and refactoring to adopt the M3 arithmetization scheme. All other source code, such as those involving the verifier, appear to be extensions of existing code written by the RISC Zero developers.

The circuit's source code contains some high-level documentation in the form of a README, but otherwise only has a limited amount of inline code documentation. The RISC Zero developers provided additional documentation in the form of whitepapers, links to relevant documentation sites, and live code walk-throughs, which the Veridise security analysts found to be helpful.

The source code contains a comprehensive test suite, including end-to-end and integration tests that cover witness generation and verification of C programs within the zkVM. It also includes the test cases contained in the official RISC-V unit test suite.

Summary of Issues Detected. The security assessment uncovered 14 issues, 5 of which are assessed to be of high or critical severity by the Veridise analysts. These issues include soundness problems affecting division instructions (V-R0M3-VUL-001, V-R0M3-VUL-002) and BigInt instructions (V-R0M3-VUL-004, V-R0M3-VUL-005). The Veridise analysts also identified 1 medium-severity issue involving behavior of the `JALR` instruction on bit 0 (V-R0M3-VUL-006); as well as 3 low-severity issues, 4 warnings, and 1 informational finding.

All 14 issues have been acknowledged by RISC Zero. Some implementation details initially identified as issues have been determined to be intended behavior after discussions with RISC Zero. These can be found in Appendix A. Of the 14 acknowledged issues, RISC Zero has fixed 10 issues. This includes all 5 critical issues. RISC Zero does not plan to fix the other 4 acknowledged issues at this time.

[§] Technical details related to the verifier math can be found here: `https://www.binius.xyz/blueprint/math`

Recommendations. After conducting the assessment of the protocol, the security analysts had a few suggestions to improve the RISC Zero M3 zkVM.

Expand Test Coverage. Due to the high number of soundness bugs found during the audit, particularly around the BigInt programs, the Veridise analysts recommend expanding the test suite to cover scenarios involving witnesses that are supposed to fail to verify. For example, to ensure that issues such as V-R0M3-VUL-002 cannot occur, a valid witness could be generated from an execution that divides some integer by 0, and then that witness could be transformed into an invalid one by setting the quotient to some arbitrary value. These mutation tests would be particularly valuable for the custom `ECALL` intrinsics, which are not covered by the official RISC-V unit test suite.

Increase Robustness of Picus Annotations. The circuit source code contains an integration with the Picus tool, in which members of a block are annotated as "inputs" or "outputs" so that Picus can be used to check determinism properties. However, Picus does not have any information regarding the values used to construct arguments. The Veridise analysts recommend using Picus to also check simple invariants, such as bounds on register values, or invariants on arguments. This would allow the developers to formally prove the absence of issues that are similar to V-R0M3-VUL-008 and V-R0M3-VUL-010. For more detailed information on the Picus annotations, see Chapter 7.

Disclaimer. We hope that this report is informative but provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the system is secure in all dimensions. In no event shall Veridise or any of its employees be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the results reported here.

2 Project Dashboard

Table 2.1: Application Summary.

Name	Version	Type	Platform
`risc0/risc0`	`da8083dc`	C++, Rust	RISC-V zkVM
`risc0/zirgen`	`df6fb9dd`	C++	RISC-V zkVM

Table 2.2: Engagement Summary.

Dates	Method	Consultants Engaged	Level of Effort
Feb. 12–Mar. 27, 2026	Manual & Tools	3	21 person-weeks

Table 2.3: Vulnerability Summary.

Name	Number	Acknowledged	Fixed
Critical Severity Issues	5	5	5
High-Severity Issues	0	0	0
Medium-Severity Issues	1	1	1
Low-Severity Issues	3	3	1
Warning-Severity Issues	4	4	3
Informational-Severity Issues	1	1	0
TOTAL	14	14	10

Table 2.4: Category Breakdown.

Name	Number
Underconstrained Circuit	6
Data Validation	4
Logic Error	2
Denial of Service	1
Maintainability	1

© 2026 Veridise Inc.

3 Security Assessment Goals and Scope

3.1 Security Assessment Methodology & Scope

The security assessment of R0VM M3 involved the following process:

1. Gathering an initial understanding of the project's business logic, users, and workflows by reviewing the provided documentation, reading the audit reports for previous versions of the project, and consulting the developers for additional information.
2. Formulating a list of possible attack vectors and system invariants.
3. Systematically reviewing the codebase for potential issues, with a focus on soundness issues and deviations from the RISC-V specification.

Attack Vectors When reviewing the codebase, the Veridise analysts searched for bugs and potential attack vectors by answering questions such as:

- Are all of the instruction implementations consistent with the RISC-V specification?
- Are unaligned addresses handled correctly by the decoder?
- Are unaligned addresses handled correctly by the control-flow instruction blocks?
- Does the M3 circuit decode instructions correctly?
- Does each block correctly ensure that "output" values are deterministic?
- Are all registers and sub-blocks properly constrained?
- Are operands to arithmetic operations such as addition and multiplication properly bounded by constraints?
- Do the `VerifyContext` and `AccumWitgenContext` correctly handle BigInt programs?
- Are the constraints of each block consistent with the intended behavior?
- Are memory accesses correctly validated against the current privilege mode?

Review Methodology. To address the questions above, the security assessment involved a combination of human experts and automated program analysis & testing tools. In particular, the security assessment was conducted with the aid of the following techniques:

- *Manual review.* Security analysts performed a line-by-line review of the in-scope code and supporting documentation to evaluate design assumptions and identify logic or soundness flaws. During the security assessment, the Veridise security analysts regularly met with the R0VM M3 developers to ask questions about the code.
- *Fuzzing.* Security analysts leveraged fuzz testing to identify soundness and completeness bugs in the proof system. To do this, they forked the Arguzz fuzzing framework and adapted it to generate RISC-V guest programs compatible with the RISC Zero zkVM. Further discussion about the fuzzing goals and methodology can be found in in Chapter 6.
- *Formal verification.* Security analysts used Picus to formally verify various properties of the zkVM zero-knowledge circuit, such as determinism and value bounds. Picus uses SMT solvers to produce a formal guarantee that such properties hold, eliminating the risk of under-constrained circuits and other critical issues. Picus was run using the AuditHub platform. Detailed information about the formal verification goals and methodology is described in Chapter 7.

© 2026 Veridise Inc.

Scope The scope of this security assessment is limited to the following files, folders, and functions of the `risc0/risc0` repository:

1. `risc0/circuit/rv32im-m3-sys/cxx/rv32im/argument`
2. `risc0/circuit/rv32im-m3-sys/cxx/rv32im/base`
3. `risc0/circuit/rv32im-m3-sys/cxx/rv32im/circuit`
4. `risc0/circuit/rv32im-m3-sys/cxx/rv32im/witness`
5. `risc0/circuit/rv32im-m3-sys/cxx/compiler/bootstrap`
6. `risc0/circuit/rv32im-m3-sys/cxx/compiler/emitter`
7. `risc0/circuit/rv32im-m3-sys/cxx/compiler/extractor`
8. `risc0/circuit/rv32im-m3-sys/cxx/verify/info`
9. Functions imported by `risc0/circuit/rv32im-m3-sys/cxx/verify/info` inside of:
    a) `risc0/circuit/rv32im-sys/cxx/rv32im/emu/blocks.cpp`
    b) `risc0/circuit/rv32im-sys/cxx/zkp/taps.cpp`
    c) `risc0/circuit/rv32im-sys/cxx/rv32im/emu/blocks.h`
    d) `risc0/circuit/rv32im-sys/cxx/rv32im/emu/poly.cpp`
10. `risc0/circuit/rv32im-m3/src/lib.rs`
11. `risc0/circuit/rv32im-m3/src/verify.rs`
12. `risc0/zkp/src/verify/mod.rs::verify_v3`

The following functions from the `risc0/zirgen` repository were also included in the scope:

1. `zirgen/circuit/predicates/predicates.cpp::ReceiptClaim::fromRv32imV3`

During the security assessment, the Veridise security analysts referred to files that were not in scope, under the assumption that such files have been implemented correctly.

3.2 Classification of Vulnerabilities

When Veridise security analysts discover a possible security vulnerability, they must estimate its severity by weighing its potential impact against the likelihood that a problem will arise.

The severity of a vulnerability is evaluated according to the Table 3.1.

Table 3.1: Severity Breakdown.

	Somewhat Bad	Bad	Very Bad	Protocol Breaking
Not Likely	Info	Warning	Low	Medium
Likely	Warning	Low	Medium	High
Very Likely	Low	Medium	High	Critical

The likelihood of a vulnerability is evaluated according to the Table 3.2.

Table 3.2: Likelihood Breakdown

Not Likely	A small set of users must make a specific mistake
Likely	Requires a complex series of steps by almost any user(s) - OR - Requires a small set of users to perform an action
Very Likely	Can be easily performed by almost anyone

The impact of a vulnerability is evaluated according to the Table 3.3:

Table 3.3: Impact Breakdown

Somewhat Bad	Inconveniences a small number of users and can be fixed by the user
Bad	Affects a large number of people and can be fixed by the user - OR - Affects a very small number of people and requires aid to fix
Very Bad	Affects a large number of people and requires aid to fix - OR - Disrupts the intended behavior of the protocol for a small group of users through no fault of their own
Protocol Breaking	Disrupts the intended behavior of the protocol for a large group of users through no fault of their own

4 Trust Model

4.1 Operational Assumptions

In addition to assuming that any out-of-scope components behave correctly, Veridise analysts assumed the following properties held when modeling security for R0VM M3.

1. The kernel program (i.e., the executable code loaded into privileged memory) is assumed to be trusted. In particular, the security assessment was performed under the assumption that the kernel is a blackbox, and that all behavior performed by the kernel is correct and intended behavior, even if the kernel has instructions that directly write to the memory locations of the register file or otherwise perform actions that interfere with the zkVM.
2. The zkVM will only be executed using user and machine privilege modes. Based on a discussion between the Veridise analysts and the developers during the time of the security assessment, other privilege modes such as supervisor mode are not planned to be supported.

© 2026 Veridise Inc.

5 Vulnerability Report

This section presents the vulnerabilities found during the security assessment. For each issue found, the type of the issue, its severity, location in the code base, and its current status (i.e., acknowledged, fixed, etc.) is specified. Table 5.1 summarizes the issues discovered:

Table 5.1: Summary of Discovered Vulnerabilities.

ID	Description	Severity	Status
V-R0M3-VUL-001	Underconstrained UnitDivBlock	Critical	Fixed
V-R0M3-VUL-002	Underconstrained DivBlock due to . . .	Critical	Fixed
V-R0M3-VUL-003	Underconstrained P2StepBlock	Critical	Fixed
V-R0M3-VUL-004	Underconstrained cycleCount in . . .	Critical	Fixed
V-R0M3-VUL-005	Consecutive BigInt rows not guaranteed . . .	Critical	Fixed
V-R0M3-VUL-006	JALR target computation does not clear bit . . .	Medium	Fixed
V-R0M3-VUL-007	Missing validation of unaligned, odd- . . .	Low	Fixed
V-R0M3-VUL-008	User mode can decode full instruction . . .	Low	Acknowledged
V-R0M3-VUL-009	BigIntBlock::verify does not check valid . . .	Low	Acknowledged
V-R0M3-VUL-010	AddressVerify::verify isMM value is non- . . .	Warning	Acknowledged
V-R0M3-VUL-011	DecodeBlock::set uses an inconsistent . . .	Warning	Fixed
V-R0M3-VUL-012	BigIntBlock::applyInner does not apply . . .	Warning	Fixed
V-R0M3-VUL-013	Claim decoding panics on invalid seal data	Warning	Fixed
V-R0M3-VUL-014	Minor code quality and maintainability issues	Info	Acknowledged

© 2026 Veridise Inc.

5.1 Detailed Description of Issues

5.1.1 V-R0M3-VUL-001: Underconstrained UnitDivBlock

Severity	Critical	**Commit**	da8083d
Type	Underconstrained Circuit	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/units.ipp:221-248`		
Confirmed Fix At	risc0/pull/3711		

Description The UnitDiv block implements 32-bit integer division. In the common case, the circuit takes as input a numerator n and denominator d and returns a quotient q and remainder r such that the following constraints hold:

1. $n = q \cdot d + r$
2. $|r| < |d|$
3. $sign(r) = sign(n)$.

The circuit implements this logic by deriving the absolute values of the numerator, denominator, quotient, and remainder and asserts that $|n| = |q| \cdot |d| + |r|$ (via 32-bit arithmetic) and then enforces the other two constraints. The correctness of the implementation relies on the expression $|q| \cdot |d| + |r|$ not overflowing when performing 32-bit arithmetic, but the circuit does not enforce that.

We attempted to verify the determinism circuit with Picus and it found the following spurious solution to the constraints:

1. `numerator = 32768`
2. `denominator = 67108608`
3. `rem = 49147`
4. `quot = 64`

As we see from this solution, `quot * denominator = 4294950912` which almost overflows in 32 bit arithmetic, but once we add `rem` in 32-bit arithmetic we recover the numerator, giving us a solution to the constraints which does not conform to the RISC-V specification.

Recommendation We recommend adding the following constraint which ensures the sum does not overflow.

```
1 EQZ(addTotRem.carryHigh.get());
```

Impact This issue breaks the soundness of the circuit. A malicious prover can construct a valid proof for incorrect or unintended computations by exploiting unconstrained degrees of freedom in the witness. This may allow an attacker to satisfy verification while violating the intended semantics of the program.

Developer Response The developers have applied the recommendation.

5.1.2 V-R0M3-VUL-002: Underconstrained DivBlock due to division-by-zero

Severity	Critical	**Commit**	da8083d
Type	Underconstrained Circuit	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/units.ipp:221-248`		
Confirmed Fix At	risc0/pull/3711		

Description `UnitDivBlock` specifies constraints that captures the semantics of 32-bit division as specified by the RISC-V ISA. When Picus is run on `UnitDivBlock`, it finds multiple distinct witnesses that satisfy the constraints when the denominator is zero. In particular, when dividing by zero, a malicious prover has free choice in the quotient they specify.

Recommendation Section 12.1.2 of the RISC-V specification specifies that when dividing by zero, "the quotient of division by zero has all bits set, and the remainder of division by zero equals the dividend." The circuit should be updated to enforce this.

Impact This issue breaks the soundness of the circuit. A malicious prover can construct a valid proof for incorrect or unintended computations by exploiting unconstrained degrees of freedom in the witness. This may allow an attacker to satisfy verification while violating the intended semantics of the program.

Developer Response The developers have applied the recommendation.

5.1.3 V-R0M3-VUL-003: Underconstrained P2StepBlock

Severity	Critical	**Commit**	da8083d
Type	Underconstrained Circuit	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/ecall.ipp:280`		
Confirmed Fix At	risc0/pull/3711		

Description The `P2StepBlock` performs an iteration of the Poseidon2 hash. The iterations can be internal (`is_finalize = 0`), where the step only updates the internal state of the hash function, or external (`is_finalize = 1`), where the result of the hash is decomposed into machine words `dataOut`).

Picus found the circuit was underconstrained, and the root cause is due to missing constraints in the external case. In particular, the `P2StepBlock` delegates the hash computation to other units and pulls the results. In the external case, the unit does the following:

```
call.isFinal = 1;
for (size_t i = 0; i < CELLS_DIGEST; i++) {
    uint32_t i2 = i + CELLS_DIGEST;
    call.in[i] = stateIn[i].get();
    call.data[i] = inValues[i].get();
    call.data[i2] = inValues[i2].get();
    call.out[i] = dataOut[i].data.high.get() * 0x10000 + dataOut[i].data.low.get();
}
```

The line `call.out[i] = dataOut[i].data.high.get() * 0x10000 + dataOut[i].data.low.get()` attempts to decompose the result of the delegated computation into two field elements `dataOut[i].data.high` and `dataOut[i].data.low` that are ranged constrained to be 16-bit values. However, this method of decomposition is incorrect as `dataOut[i].data.high.get() * 0x10000 + dataOut[i].data.low.get()` can overflow in BabyBear field. Picus found the following distinct decompositions for a case where `call.out[i] = 0` : `dataOut[i].data.high = dataOut[i].data.low = 0` and `dataOut[i].data.high = 61440` and `dataOut[i].data.low = 2`.

Impact This issue breaks the soundness of the circuit. A malicious prover can construct a valid proof for incorrect or unintended computations by exploiting unconstrained degrees of freedom in the witness. This may allow an attacker to satisfy verification while violating the intended semantics of the program.

Recommendation We recommend performing additional range checks on `dataOut[i].data.high` to ensure the sum cannot overflow the field, as is done in the V2 circuit here: https://github.com/risc0/zirgen/blob/main/zirgen/circuit/rv32im/v2/dsl/inst_p2.zir#L280.

Developer Response The developers fixed this issue by removing the `dataOut` array from the block.

5.1.4 V-R0M3-VUL-004: Underconstrained cycleCount in EcallBigIntBlock

Severity	Critical	**Commit**	da8083d
Type	Underconstrained Circuit	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/ecall.ipp:398-411`		
Confirmed Fix At	risc0/pull/3732, 2e2c8b57		

Description In the m3 circuit implementation of RISC-V, the `ECALL` instruction can be used with register A7 set to the value `5` to execute a sequence of "BigInt instructions". This particular feature is implemented in the `EcallBigIntBlock` component, which will decode the instruction and add appropriate `CpuStateArgument` and `BigIntCpuStateArguments` to "launch" the execution of the BigInt instructions.

The number of BigInt instructions to be executed is determined by a `cycleCount` register stored in the `EcallBigIntBlock` component, but there are no constraints on `cycleCount` that relate it to the memory state.

```
1  Val<C> cycleVal = cycle.get();
2  Val<C> countVal = cycleCount.get();
3  Val<C> biPc = pcDecomp.wordAddr(readT2.data.get());
4  // ...
5  BigIntCpuStateArgument<C> initialBigIntState(cycleVal + 1, biPc, mm.get());
6  BigIntCpuStateArgument<C> finalBigIntState(cycleVal + countVal + 1, biPc + countVal
     , mm.get());
7  ctx.push(initialBigIntState);
8  ctx.pull(finalBigIntState);
9  BIGINT_STATE_ARGUMENT(ctx, initialBigIntState, finalBigIntState);
10
11 ctx.push(CpuStateArgument<C>(
12     cycleVal + countVal + 2, fetch.nextPc.get(), MODE_MACHINE, fetch.iCacheCycle.
     get()));
```

Snippet 5.1: Relevant lines of code in `EcallBigIntBlock::addArguments` where the `cycleCount` is used to determine how many BigInt instructions to execute.

Impact The `cycleCount` is effectively independent of the program being executed, so a prover can arbitrarily choose any `cycleCount` desired when generating a witness. Consequently, for a given initial memory state, a prover is able to arbitrarily "terminate" or "extend" a BigInt instruction sequence, resulting in BigInt computations that may not be consistent with the intended behavior of the program. Notably, BigInt instructions can write to memory locations and read from registers (including privileged memory, if running in machine mode), so exploiting the BigInt instruction count may allow a malicious prover to arbitrarily access memory or bypass checks.

Additionally, if `cycleCount` is chosen to be a large integer, then an integer overflow could occur when performing the addition operations used to compute the CPU cycle values passed to the final `BigIntCpuStateArgument` and the pulled `CpuStateArgument`. When this happens, the resulting CPU cycle values will be strictly less than the CPU cycle of the BigInt `ECALL` and coincide with an instruction executed at a previous cycle. An attacker could potentially use this coincidence as part of a larger exploit.

Recommendation Change the BigInt `ECALL` format to designate a register to represent the `cycleCount`, and ensure that the `cycleCount` is constrained to be the value retrieved from a read of the designated register.

Developer Response The developers have modified `ECallBigIntBlock` so that register A0 is required to store a "descriptor address", where the cycle count is read from the memory word following that of the descriptor address. Additionally, the cycle count is required to be strictly less than 2^{16}.

5.1.5 V-R0M3-VUL-005: Consecutive BigInt rows not guaranteed to be of consecutive cycles

Severity	Critical	**Commit**	da8083d
Type	Underconstrained Circuit	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/circuit/top.ipp:91`		
Confirmed Fix At	risc0/pull/3736, 0e3b0fd0		

Description Because BigInt instructions are implemented using a finite field extension `FpExt`, they must be handled separately by the m3 circuit. Specifically, each row in the computational trace table is associated with an `AccumTop` data structure that holds state related to any BigInt operations performed in the row. If the row does not perform a BigInt operation, the row's `AccumTop` should hold a `polyOp` value of `NOP`. Otherwise, the row's `AccumTop` should hold state that relates to the data in the `BigIntBlock` and the state in the `AccumTop` of the BigInt operation executed in the previous cycle.

In the current implementation of `AccumTop`, the previously executed BigInt operation is assumed to have occurred in the previous row. However, neither the table structure nor the `AccumTop::verify` method enforces any cycle ordering properties on the rows. There is, consequently, no guarantee that the `AccumTop` from the previous row corresponds to the BigInt operation executed in the previous cycle.

Impact A malicious prover could reorder rows such that the previous row has `polyOp` value of `NOP`, allowing the values for the current row to be chosen arbitrarily. Every `NOP` will reset the `term, poly, total` values of the `AccumTop` state, regardless of the current state. This can be exploited to change the result of BigInt computations or to bypass `EQZ` operations used in constraints involving BigInts.

As an example, consider a sequence of rows whose `polyOps` are `NOP op1 op2 EQZ NOP`. This could be reordered as `NOP op1 op2 NOP EQZ`. The second `NOP` will reset the the `AccumTop` state, so that the prover could choose `local = 0` to pass the `EQZ` constraint.

Recommendation Change the `AccumTop` data structure to record cycle information, and update the `AccumTop::verify` method to validate that the `cycle` is equal to 1 plus the `cycle` of the previous row's `AccumTop`. Additional constraints may also be required in order to correctly handle any preceding/following rows that do not select the BigInt block type.

Developer Response The developers have applied the recommendation. Specifically, the `BigIntBlock` now has a `first` register indicating whether it is the "first" instruction executed in a BigInt program. This is set by the `BigIntCpuStateArgument` passed by `EcallBigIntBlock`. The `AccumTop` now has a `biCycle` register which is constrained to be equal to the current `cycle` of the `BigInt` block (if selected in the row), or 0 otherwise. This `biCycle` is used to ensure that when the `BigInt` block is selected and the `first` register of the BigInt block is 0, then `biCycle = prev->biCycle + 1`.

5.1.6 V-R0M3-VUL-006: JALR target computation does not clear bit 0 before next-PC transition

Severity	Medium	**Commit**	da8083d
Type	Data Validation	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/circuit/` ▶ `inst.h:402` ▶ `inst.ipp:626-627`		
Confirmed Fix At	risc0/pull/3732, 2e2c8b57		

Description The RISC-V specification, Section 2.6.1, indicates that the `JALR` instruction must compute the target address "by adding the sign-extended 12-bit I-immediate to the register *rs1*, then setting the least-significant bit of the result to zero." This ensures a global invariant that the program counter is always an even number (i.e., aligned to a 2-byte boundary).

However, in the `InstJalrBlock`, the next program counter address is computed as `rs1 + imm`, without the required low-bit clearing. Thus, if either operand in the sum is odd, it would be possible to change the program counter to an odd number (i.e., not aligned to a 2-byte boundary).

```
// In InstJalrBlock::applyInner:
T::apply(ctx, "sumPc", sumPc, readRs1.data.get(), imm.get());   // sumPc = rs1 + imm

// In InstJalrBlock::addArguments:
ctx.push(CpuStateArgument<C>(cycleVal + 1, sumPc.get(), mode, fetch.iCacheCycle.get()
    ));  // nextPc = sumPc
```

Snippet 5.2: *Relevant lines in* `InstJalrBlock::applyInner()` *and* `InstJalrBlock::addArguments()` *in inst.ipp that calculate and propagate the raw sum, without the low-bit clearing.*

Impact This issue affects the implementation of both the prover and verifier. If a JALR instruction is executed such that `rs1 + imm` is odd, the program counter will change to an odd number. This violates the RISC-V specification. There is only limited impact from having an odd program counter; see V-R0M3-VUL-011 for further details.

Recommendation

- Change the `InstJalrBlock` implementation to compute the target address as `(rs1 + imm) & ~1`, and update both `verify()` and `addArguments` to use this expression as the next program counter.
- Implement the corresponding changes in the emulator.

Developer Response The developers have changed `InstJalrBlock` to be compliant with the RISC-V specification. The changes include a new `mid15` register which is constrained to be equal to bits 15-1 of the computed address. The value `mid15 * 2` is used as the next program counter, which ensures that the low bit is cleared.

5.1.7 V-R0M3-VUL-007: Missing validation of unaligned, odd-numbered program counter

Severity	Low	**Commit**	da8083d
Type	Data Validation	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/decode.ipp:178-267`		
Confirmed Fix At	risc0/pull/3732, 2e2c8b57		

Description Section 2.6.1 of the RISC-V specification implicitly suggests that when compressed instructions are supported, there is a global invariant that the program counter must always be an even number.

> Instruction-address-misaligned exceptions are not possible on machines that support extensions with 16-bit aligned instructions, such as the compressed instruction-set extension, C.

However, the `DecodeBlock` does not fully enforce this invariant. Specifically, the lower 16 bits of the `pc` register (containing the program counter) are split into three parts using the `AddressDecompose` component: `low0`, `low1` and `mid14` bits. Although the `low1` bit is checked to determine whether the instruction should be loaded from the middle or start of a word, the `low0` bit is not constrained at all. This allows the `pc` to assume mis-aligned, odd values.

```
1
2 // rv32im/circuit/decode.ipp
3 Val<C> instWordAddr = pcDecomp.wordAddr(fetch.pc.get());
4 Val<C> isUnaligned = pcDecomp.low1.get();
5 // no EQZ(pcDecomp.low0.get()) style constraint to ensure alignment
```

Snippet 5.3: Relevant lines in `DecodeBlock::verify()`

Impact In most cases, the `pc` should be an even number, because it is initialized as one, incremented by even numbers, and set to even numbers by the branch instructions. However, bugs such as V-R0M3-VUL-006 may allow the `pc` to be set to an odd value. Such a `pc` value will cause the instruction to be loaded from the same memory index as the same `pc` rounded down to an even number. This occurs because the memory index is calculated by bit-shifting the address to the right, causing the `low0` bit to be ignored.

Furthermore, when handling odd-valued `pc`s, the behavior in the verifier is inconsistent with that of the prover, as described in V-R0M3-VUL-011. Due to this issue, a witness generated by an honest prover might not be verified.

However, outside of these minor differences and the deviation from the RISC-V specification, the Veridise auditors did not find any concrete exploits that could result from the existence of odd-valued program counters.

Recommendation

- Fix any related issues that allow the program counter to be set to an odd value.
- Consider adding a constraint in `DecodeBlock::verify`, such as `pcDecomp.low0 == 0`, that prevents the verifier from accepting an odd-valued program counter.

Developer Response The developers have applied the recommendation.

5.1.8 V-R0M3-VUL-008: User mode can decode full instruction partially contained in kernel memory

Severity	Low	**Commit**	da8083d
Type	Data Validation	**Status**	Acknowledged
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/decode.ipp:185-188`		
Confirmed Fix At	N/A		

Description To handle a full instruction word loaded at a 2-byte alignment boundary, the `DecodeBlock` will load both the word located at the program counter `fetch.pc` and the subsequent word. However, only the `fetch.pc` is validated against the current privilege mode (via the `AddressVerify` component); the subsequent word is not. Thus, when a full instruction word starts at the last 2 bytes in user mode memory, the `DecodeBlock` is constrained such that the low bits of the instruction are read from user space, but the high bits may be read from protected memory reserved for the kernel.

For example, consider a witness with a cycle where a full instruction word is loaded from `fetch.pc = 0xBFFFFFFE`. The `DecodeBlock` would be required to make two loads, one to the memory index corresponding to the address `0xBFFFFFFC` and one to `0xC0000000`. The latter is contained in kernel memory; however, the constraints generated by the `AddressVerify` component are satisfiable because `0xBFFFFFFC` is contained in user memory.

```
1 // rv32im/circuit/decode.ipp
2 Val<C> instWordAddr = pcDecomp.wordAddr(fetch.pc.get());
3 Val<C> isUnaligned = pcDecomp.low1.get();
4 Val<C> load1Addr = cond<C>(isCompressed.get(),
5                            Val<C>(COMPRESSED_INST_LOOKUP_WORD) + low16(),
6                            cond<C>(isUnaligned, instWordAddr + 1,
      COMPRESSED_INST_LOOKUP_WORD));
7 EQ(load1Addr, load1.wordAddr.get());
```

Snippet 5.4: Lines in `DecodeBlock::verify` that constrain the addresses.

Impact Under ordinary scenarios, this issue should have no impact. Even if the instruction at the address `0xBFFFFFFE` is decoded and executed in user mode, the next instruction will be contained in kernel memory. A witness that includes decoding at the next cycle cannot be verified, since it would be unable to satisfy the privilege mode constraint imposed by the `AddressVerify` component on `fetch.pc`.

However, if the instruction located at `0xBFFFFFFE` is a jump or branching instruction, it could potentially be exploited by an attacker. For example, if the guest program itself contains memory-related vulnerabilities, an attacker could use those vulnerabilities along with the bug described in this issue to perform unintended behavior with respect to the guest program.

Recommendation Add a constraint to ensure that if a full instruction word is loaded from the "middle" of a word, the second word's address will be checked against the current privilege mode.

Developer Response The developers acknowledge that the issue can occur but do not plan to address it due to the limited impact:

> The impact of this is limited to at worst allowing a malicious guest program to read the first 2 bytes of kernel memory. In real world uses that region of kernel memory is in the text segment and all parties know the kernel, this is not practically exploitable and correcting it would add meaningful additional circuit complexity.

5.1.9 V-R0M3-VUL-009: BigIntBlock::verify does not check valid polyOps with CHECK memOp

Severity	Low	**Commit**	da8083d
Type	Underconstrained Circuit	**Status**	Acknowledged
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/bigint.ipp:50`		
Confirmed Fix At	N/A		

Description In the witness generator for BigInt instructions, the `BigIntPreflight::step` function ensures that when the `memOp` is set to `CHECK`, the `polyOp` is required to be one of `CARRY_1, CARRY_2, SHIFT, EQZ`. However, there is no constraint enforcing this property in `BigIntBlock::verify`.

```
1  if (inst.polyOp != static_cast<uint32_t>(PolyOp::NOP) &&
2      inst.memOp == static_cast<uint32_t>(MemoryOp::CHECK)) {
3    LOG(2, "MemoryOp::CHECK && !PolyOp::NOP");
4    // ...
5    for (size_t i = 0; i < BIGINT_WIDTH_BYTES; i++) {
6      uint32_t val = totCarry[inst.offset * BIGINT_WIDTH_BYTES + i] + basePoint;
7      switch (PolyOp(inst.polyOp)) {
8      case PolyOp::CARRY_1:
9        ret[i] = (val >> 14) & 0xff;
10       break;
11     case PolyOp::CARRY_2:
12       ret[i] = (val >> 8) & 0x3f;
13       break;
14     case PolyOp::SHIFT:
15     case PolyOp::EQZ:
16       ret[i] = val & 0xff;
17       break;
18     default:
19       throw std::runtime_error("Invalid polyOp with MemoryOp_Check");
20     }
21   }
```

Snippet 5.5: Lines of code in `BigIntPreflight::step` that computes the in the case of `memOp` set to `CHECK`. Note that `CARRY_1, CARRY_2, SHIFT, EQZ` are the only `polyOps` allowed in this case.

Impact A malicious prover could generate a proof for an execution trace involving BigInt instructions that set `memOp` to `CHECK`, but with `polyOp` to `SET_TERM` or `ADD_TOTAL`. Because the `BigIntBlock` does not constrain its `bytes` member when the `memOp` is `CHECK`, this would allow the prover to manipulate the `AccumTop`'s `term` value set in `SET_TERM`, or the `total` value that will be set in `ADD_TOTAL`.

In most cases, provers will use the witness generator that comes with the m3 source code, in which case an error will be thrown when attempting to execute BigInt instructions with the disallowed `polyOp` combinations.

Recommendation Consider adding a constraint in `BigIntBlock::verify` that restricts the set of valid `polyOp` values when the `memOp` is `CHECK`.

Developer Response The developers acknowledge the issue, but state that:

> This will only impact malicious guest programs that run invalid / illegal BigInt programs. We do not specify behavior of non-conforming BigInt programs.

5.1.10 V-R0M3-VUL-010: AddressVerify::verify isMM value is non-boolean for supervisor mode

Severity	Warning	**Commit**	da8083d
Type	Data Validation	**Status**	Acknowledged
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/u32.ipp:62-77`		
Confirmed Fix At	N/A		

Description The `AddressVerify` component is used to enforce that a given address value cannot be a "protected" address when operating outside of the machine privilege mode. This enforcement is implemented as an equality constraint in `AddressVerify::verify`, where a value `isMM = mode * inv(MODE_MACHINE)` is computed to determine whether the machine mode check should be applied. Specifically, `isMM` determines the highest legal `topAddr` of the current privilege mode, subtracts the address's high limb `val.high` from it, and compares the result to the value `highSub`. The value of `highSub` is constrained to be a 16-bit integer; thus, `highSub` only exists when `topAddr` is greater than or equal to `val.high`.

```
1 Val<C> isMM = mode * Val<C>(inv(Fp(MODE_MACHINE)));
2 Val<C> topAddr = cond<C>(isMM, 0xffff, 0xbfff);
3 EQ(topAddr - val.high, highSub.get());
```

Snippet 5.6: Lines in `AddressVerify::verify` that generate the constraint.

The intended behavior is for `isMM` to be a boolean value, as the `cond()` helper function computes `topAddr` as a linear combination with `isMM` and `1 - isMM` as coefficients. However, when `mode` is `MODE_SUPERVISOR`, then `isMM` can be a value greater than 1, so `topAddr = cond(isMM, 0xffff, 0xbfff)` will not be the values `0xffff` or `0xbfff`. This could potentially allow the constraint to be satisfied for a protected address even if the mode is not `MODE_MACHINE`.

Impact In practice, in the current prime field, it is not possible to construct a witness with a protected address and `mode = MODE_SUPERVISOR` that satisfies the equality constraint, as the computed `topAddr` will be so large that subtracting `val.high` will always produce a result that is larger than a 16-bit integer (so that no valid `highSub` value exists). We have also checked that this is the case using Picus.

Recommendation Update the `isMM` constraints to correctly ensure that it is required to be 0 when `mode` is set to `MODE_SUPERVISOR`. For example, this could be accomplished by using an `IsZero` component on `mode - MODE_MACHINE` to constrain the value of `isMM`.

Developer Response The developers have acknowledged the issue but note that it is not a problem in the current implementation:

> Supervisor mode is not currently supported in the circuit (and is reserved only for future support of S-mode), and can never be set, thus this condition is 'unreachable'.

5.1.11 V-R0M3-VUL-011: DecodeBlock::set uses an inconsistent unaligned-PC predicate

Severity	Warning	**Commit**	da8083d
Type	Logic Error	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im-sys/cxx/[...]/decode.ipp:126, 132-140`		
Confirmed Fix At	risc0/pull/3732, 2e2c8b57		

Description `DecodeBlock::set` checks whether the program counter is aligned to a 32-bit boundary using the expression `pc % 4 == 2`. This is used to determine whether the instruction word is loaded from the "start" of the memory word corresponding to `pc` (when `pc` is a multiple of 4 bytes) or the "middle" of the memory word (when `pc % 4 == 2` is true).

```
uint32_t low16 = witness.fetch.pc % 4 == 2 ? baseInst >> 16 : baseInst & 0xffff;
// ...
if (compressed) {
  inst = witness.load1.value;
} else {
  if (witness.fetch.pc % 4 == 2) {
    inst = witness.load0.value >> 16 | witness.load1.value << 16;
  } else {
    inst = witness.load0.value;
  }
}
```

Snippet 5.7: Snippet from `DecodeBlock::set` showing where the expression `pc % 4 == 2` is used.

However, the behavior in `DecodeBlock::set` diverges from that of `DecodeBlock::verify` when `pc` is of the form `4n+3`. In such cases, `DecodeBlock::set` considers such `pc`s to correspond to the "start" of the word, whereas `DecodeBlock::verify` considers such `pc`s to be loaded from the "middle" of the word:

```
FDEV Val<C> low16() DEV {
  return cond<C>(pcDecomp.low1.get(), load0.data.high.get(), load0.data.low.get());
}

// In DecodeBlock::verify:

Val<C> isUnaligned = pcDecomp.low1.get();
// Get actual instruction data
Val<C> instLow = cond<C>(isCompressed.get(), load1.data.low.get(), low16());
Val<C> instHigh = cond<C>(isCompressed.get(),
                          load1.data.high.get(),
                          cond<C>(isUnaligned, load1.data.low.get(), load0.data.high.
    get()));
```

Snippet 5.8: Snippets from `DecodeBlock` showing how the instruction is loaded. This is determined by checking whether bit 1 of the pc is set, equivalent to `pc % 4 == 2 || pc % 4 == 3`.

Impact Due to V-R0M3-VUL-006, it is possible for a prover to construct a witness with a cycle where `pc % 4 == 3` is true. The witness will be rejected by the verifier, leading to a completeness issue.

In practice, this issue should have limited impact, as the RISC-V has an implicit invariant that program counters cannot have odd values; even if they were to occur, they would likely be the result of some other bug in the circuit.

Recommendation Change `DecodeBlock::set` so that the expression `pc % 4 == 2` is replaced with `pc % 4 >= 2`.

Developer Response The developers fixed V-R0M3-VUL-007, making this issue infeasible.

5.1.12 V-R0M3-VUL-012: BigIntBlock::applyInner does not apply prevCycle and prevValue

Severity	Warning	**Commit**	da8083d
Type	Logic Error	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/bigint.h:46-49`		
Confirmed Fix At	risc0/pull/3691, a2791a81		

Description In the `BigIntBlock`, the registers `prevCycle` and `prevValue` are not instantiated by the `applyInner` method.

Impact These registers will not be correctly tracked by the `RecordingContext`, resulting in a crash when extracting the `BigIntBlock` to `zhlt` dialect IR.

Recommendation Update `BigIntBlock::applyInner` to invoke `T::apply` on `prevCycle` and `prevValue`.

Developer Response The developers have applied the recommendation.

5.1.13 V-R0M3-VUL-013: Claim decoding panics on invalid seal data

Severity	Warning	**Commit**	da8083d
Type	Denial of Service	**Status**	Fixed
Location(s)	`risc0/circuit/rv32im/src/lib.rs:170-188`		
Confirmed Fix At	risc0/pull/3740, c0ae7ba1		

Description When a receipt is verified in the Rust bindings, the `Claim::decode` method is used to deserialize the receipt's claim from the seal data. The fields are deserialized using various "read" methods on a helper struct named `Decode`.

```
let pre_state = decoder.read_digest_from_words()?;
let post_state = decoder.read_digest_from_words()?;
let is_terminate = decoder.read_u32_from_elem()?;
let term_a0 = decoder.read_u32_from_shorts()?;
let term_a1 = decoder.read_u32_from_shorts()?;
let output = decoder.read_digest_from_shorts()?;
let mut povw_nonce = [0u32; 8];
for v in &mut povw_nonce {
    *v = decoder.read_u32_from_shorts()?;
}
```

Snippet 5.9: Lines in `Claim::decode` that read the claim fields from the seal.

Some of these read methods, including `read_u32_from_elem`, `read_u32_from_shorts`, `read_digest_from_words`, and `read_digest_from_shorts`, will use the `bytemuck::checked::cast` (and the slice variant) to perform a conversion from a read `u32` value to a finite field element. According to the `bytemuck` documentation, these cast functions will panic if an invalid finite field element is provided as input.

```
    fn read_u32_from_shorts(&mut self) -> Result<u32> {
        let slice = bytemuck::checked::cast_slice::<_, Elem>(self.read(2)?);
        let (high, low) = (slice[1], slice[0]);
        let val = (high.as_u32() & 0xffff) << 16 | (low.as_u32() & 0xffff);
        Ok(val)
    }
```

Snippet 5.10: Example of `bytemuck::checked::cast` being used in `Decoder::read_u32_from_shorts`.

Additionally, in the case of `read_u32_from_shorts`, the resulting u32 is obtained by reading two `u32` values, treating them as two 16-bit integers, and using them as the high and low limbs of the final result. No checks are performed to validate that the two values are valid `u16`s; rather, any higher bits are silently ignored.

Impact When verifying a receipt, if the claim contains an invalid finite field element, then the verifier will panic. This can lead to a denial-of-service issue if the verifier is running as an online service that is expected to have high availability.

Recommendation

- Change the `Decoder`'s read helper methods to use the `bytemuck::checked::try_cast` series of functions, which will return a `Result` instead of causing a `panic`.

- Consider adding code to `Decoder::read_u32_from_shorts` to return an error when `high` or `low` are larger than `0xffff`.

Developer Response The developers have applied the recommendation.

5.1.14 V-R0M3-VUL-014: Minor code quality and maintainability issues

Severity	Info	**Commit**	da8083d
Type	Maintainability	**Status**	Acknowledged
Location(s)	`risc0/circuit/rv32im-sys/cxx/` ▶ `compiler/` • `emitter/Emitter.cpp:35` • `extractor/extract.h:71-79` ▶ `rv32im/circuit/` • `decode.h:43` • `decode.ipp` • `globals.ipp:80` • `inst.h:44, 109` • `inst.ipp:48, 731` • `lookups.ipp` • `top.h:106`		
Confirmed Fix At	N/A		

Description During the manual review of the code, the security analysts made the following observations regarding code style, incorrect comments, and maintainability concerns.

- **Typos, incorrect names, and incorrect comments**
  - Incorrect comments or typos in the following locations may lead to developer confusion:

  1. `rv32im/circuit/inst.ipp:731` : In `InstEcallBlock::verify`, a comment states that `Make sure next PC is being saved`. However, the current PC in saved in the MEPC register (note that the `InstMretBlock` will set the program counter to the saved PC + 4).
  2. `rv32im/circuit/inst.h:109` : The `NAME` member of the `DualReg` register is set to `DestReg`.
  3. `rv32im/circuit/decode.h:43` : The comment on the `FetchBlock::isMM()` function states `Return 1 if in machine mode`; however the function `isMM()` actually returns the integer representing the privilege `mode`.
     This function should be renamed to `getMode()`.

- **Use of magic constants.**
  The analysts observed the use of integer literals throughout the code base. This is error-prone because these literals will need to be kept consistent with related code. A few examples are described below.

  1. `rv32im/circuit/inst.ipp:48` : In call to `RANGE_POSTCONDITION` macro, a literal `18` is used to provide the upper bound on the `option` field of a `UnitArgument`.
  2. `rv32im/circuit/decode.ipp` : In `DecodeBlock::verify`, when computing the value `isValid` for each block type, the literal `7` is used to decompose the value `gidx` into `idx1` and `idx2`. The `7` represents the size of the block type table.
  3. `rv32im/circuit/lookups.ipp` : In several places, `LookupArguments` are instantiated with integer IDs `0,1,2` corresponding to different lookup tables (u8 range, u16 range, iCache cycles). Additionally, the sizes of these tables are hardcoded as literals `256, 65536`.

- **Duplicate code in** `rv32im/compiler/extractor/extract.h`**.** There are multiple copies of the `extract()` function that differ only in parameters. These can be replaced with a generic templated version:

```
template <template <typename Ctx> typename Component, template <typename Ctx>
    typename ...Args, typename Func>
void extractWithArgBuilder(RecordingContext& ctx, Func &&func) {
  mlir::Type layoutType = getLayoutType<Component, Args<layoutbuilder::Context
    >...>(ctx.mlirCtx);
  ctx.enterComponent(Component<RecordingContext>::NAME, layoutType);
  std::tuple<Args<RecordingContext>...> args = func(ctx);
  Component<RecordingContext> component;
  ctx.componentIRMap = populateComponent<Component, Args<RecordingContext>...>(
    ctx, component);
  std::apply([&](auto &&... argPack) {
    AddArgumentsFwd::apply(ctx, component, argPack...);
    VerifyFwd::apply(ctx, component, argPack...);
  }, args);
  ctx.exitComponent();
}

// Example:
extractWithArgBuilder<AddressDecompose, ValU32>(ctx, [](auto &ctx) {
  auto low1 = ctx.addValParameter();
  auto high1 = ctx.addValParameter();
  return std::make_tuple(ValU32<RecordingContext>(low1, high1));
});
```

- **Potentially undefined behavior in** `AccumTop::verify`.
  On
  `risc0/circuit/rv32im-sys/cxx/rv32im/circuit/top.ipp`
  , line 106, accessing the value `top->mux.BigInt.data` is technically undefined behavior in C++ if the row does not select a `BigInt` value. However, in such a case, this should not have any impact since the value will be multiplied by 0.
- **Manual encoding of options**. In several block implementations, the `UnitArgument` options are encoded manually using multiplication/addition expressions, which is error-prone. If possible, the `EncodeOptions` helper struct in `risc0/circuit/rv32im-sys/cxx/rv32im/base/opt.h` should be used instead.
- **The cycle lookup table may be larger than** `finalCycle`.
  In `GlobalsBlock`, the cycle lookup table will have a size that is rounded up to lowest multiple of 16 greater than `finalCycle`. This may potentially allow memory read/write blocks after the final cycle, although there should be limited impact in doing so.
- **Emitter fails IR verification with** `--emit=check`.
  The emitter program in
  `risc0/circuit/rv32im-sys/cxx/compiler/emitter/main.cpp`
  fails to run with passed the `--emit=check` or `--emit=rust`. The cause is that the `GenerateCheckLayout` pass inserts `zhlt.magic` ops, which fail during MLIR op verification.

Impact While these issues do not currently introduce exploitable vulnerabilities, they increase the likelihood of future bugs and reduce the ability to reason about correctness. Over time, this can lead to inconsistent behavior, missed edge cases, and degraded security assurances.

Recommendation We recommend incrementally addressing these issues as part of ongoing development, with a focus on improving code clarity, reducing duplication, and aligning implementations with consistent patterns.

Developer Response The developers have acknowledged this issue, stating that:

> We agree there is room to improve on these issue concerns and address some of them during forward development.

6 Fuzz Testing

6.1 Methodology

One goal of the security assessment was to fuzz R0VM M3 in order to identify soundness and completeness bugs in the proof system. In this context, *soundness bugs* are executions accepted by the constraint system that do not correspond to a valid RISC-V execution, while *completeness bugs* are valid RISC-V executions that are rejected by the constraint system.

Fuzzing is particularly useful for bug classes that are not well captured by local determinism checks alone, such as incorrect memory-argument logic, witness/constraint mismatches, or constraints that are deterministic but do not encode the intended semantics. To exercise these classes of issues, the Veridise security analysts used Arguzz, a zkVM fuzzer based on metamorphic testing and targeted fault injection.

6.1.1 Arguzz Overview

At a high level, Arguzz generates a bundle of semantically equivalent guest computations and embeds them into a generated host/guest project. The guest compares the outputs of these equivalent computations on the same inputs and commits a distinguished success value when they all agree. Arguzz then performs a baseline traced execution of the generated project and uses the resulting instruction trace to select a targeted fault injection site and injection kind. The same program is then re-executed with a single injected fault, and the baseline and injected executions are compared.

This methodology is useful for identifying both completeness and soundness issues. Completeness bugs appear when the baseline execution of a valid program fails to prove. Soundness bugs appear when a perturbed execution still produces a successful proof, or when the trace and committed outputs are inconsistent with the intended semantics after injection. Because Arguzz reasons over whole executions rather than isolated constraint blocks, it can expose issues that are missed by more local analyses.

6.1.2 Adaptation to the RISC Zero M3 Backend

Arguzz did not natively target the current RISC Zero M3 backend used in this assessment, so the harness was adapted to the current M3 codebase before fuzzing. In particular, the analysts ported the instruction taxonomy and fault-injection hooks to the current `rv32im` execution path, updated the source-rewriting logic to the current repository layout, and preserved fault-injection state across emulator steps so that injections could be scheduled reliably against traced execution steps. The generated projects were also updated to build and run against the current M3 workspace structure.

These changes were engineering work needed to make the fuzzer applicable to the current M3 line. After this adaptation, the harness was able to build generated projects, execute baseline traces, and run single-fault injection campaigns against the current M3 implementation.

© 2026 Veridise Inc.

6.1.3 Campaign Configuration

The Arguzz campaigns were executed on a Google Cloud `n4-standard-32` instance with 32 vCPUs and 128 GB of RAM for approximately 72 hours. The campaigns used a persistent disposable clone of the target repository together with persistent build and cache directories so that generated projects and native artifacts could be reused across iterations.

6.1.4 Coverage and Limitations

The resulting campaigns primarily exercised the core RV32IM instruction classes implemented by the M3 circuit, including integer arithmetic and logic, immediate operations, shifts, multiply/-divide/remainder instructions, conditional branches, jumps, loads, and stores. The campaigns also exercised general control/system instructions such as `ecall` and `mret`. The enabled fault-injection strategies included program-counter perturbations, instruction-word modification, branch-condition negation, computed-result perturbation, load/store value perturbation, and pre-/post-execution register and memory modifications.

This coverage should not be interpreted as exhaustive. In particular, the campaigns did not comprehensively target all RISC Zero-specific precompile or accelerator paths. We recommend that future work extend the harness to cover specialized functionality such as the BigInt instructions and the Poseidon hash path. These components are security-relevant, and issues identified elsewhere in the review indicate that they are worthwhile targets for further fuzzing.

6.1.5 Results

The Arguzz campaigns described above did not identify any soundness or completeness bugs in the portions of the M3 backend exercised by our generated test cases and fault-injection strategies.

As with all fuzzing-based evaluations, this result should be interpreted as evidence about the tested configurations and exercised execution paths rather than as a proof of correctness. The absence of findings in these campaigns does not rule out bugs in instruction paths, precompiles, or auxiliary mechanisms that were not adequately covered by the generated workloads.

7 Formal Verification

As part of the security assessment of the R0VM M3, the Veridise analysts used the Picus tool to formally verify various properties of the zkVM's zero-knowledge circuit. This section describes the methodology, properties validated, and follow-up recommendations.

7.1 Background

The RISC Zero developers integrated the R0VM M3 circuit source code with Picus by creating an "extractor" program that converts a given set of blocks into Picus modules containing their constraints. Each parameter of a block will be marked as an input, while each member and subcomponent of the block will be marked as an output. The constraints will be converted into Picus `assert` commands.

The main goal of this integration is to guard against vulnerabilities arising from circuit non-determinism. Given a circuit with a set of input parameters, the circuit is said to be *deterministic* if for each combination of input parameters, there exists at most one witness with those input parameters. A deterministic circuit is less likely to exhibit under-constrained circuit bugs and therefore soundness issues.

When run on a circuit, Picus may return one of three results: verified, counterexample, or unknown. A result of verified means that Picus was able to mathematically prove that the circuit is deterministic. If Picus is instead able to show that the circuit is non-deterministic, it will produce a counterexample consisting of two witnesses that have the same input parameters. In other cases, the circuit may involve complicated constraints where determinism or non-deterministm cannot be confirmed within a reasonable amount of time; in this case Picus will report "unknown".

The circuit code uses C++ macros, which we refer to as "Picus annotations", in order to provide additional information to Picus. These annotations include:

- `PICUS_INPUT(x)` adds an assumption that the value `x` is deterministic.
- `PICUS_ARGUMENT(inputs, outputs)` adds an assumption that if the list of `inputs` are deterministic, then the `outputs` must also be deterministic.
- `RANGE_PRECONDITION(lb, x, ub)` adds two assumptions `lb <= x` and `x < ub`.
- `RANGE_POSTCONDITION(lb, x, ub)` adds two constraints `lb <= x` and `x < ub`. These should not be confused with Picus post-condition commands.

7.2 Methodology

The formal verification aspect of this security assessment involved two tasks: to validate the determinism annotations in the circuit source code, and to check for additional functional correctness properties that could be validated with Picus. The Veridise security analysts carried out these tasks while performing the manual code review. In particular, whenever they reviewed the source code of a block or component, they ran the extractor program to generate the corresponding Picus file, and then they used Picus to validate the determinism of the block or

 © 2026 Veridise Inc.

component. The analysts then inspected the Picus annotations to check if the annotations were consistent with the constraints. In addition to determinism, when the behavior of a component or block was not clear from reading the constraints, the analysts also used Picus to check for additional functional correctness properties.

A comprehensive list of the reviewed properties, as well as follow-up recommendations, are described in the subsequent sections.

7.3 Properties Validated

The Veridise analysts validated several categories of properties of the circuit's components during the security assessment. The main focus was on determinism properties, with some additional validation of other functional correctness properties.

7.3.1 Determinism of Components

Table 7.1 shows the components that were checked for determinism properties by the Veridise analysts. The "Verified Deterministic" column indicates the result of running Picus on the component. It is marked with "Yes" if Picus could verify the determinism of the block. If additional annotations had to be inserted by the security analysts to make the block verify, the column is marked "With ..." followed by the required change. Otherwise, the block could not be proven to be deterministic and is marked with "No". Picus was run with the `z3` solver, unless otherwise stated.

Some components, such as PhysMemReadBlock, are intentionally non-deterministic because they are primarily used as subcomponents of other components or blocks. The determinism of these components' output values are instead determined by the constraints imposed by their parents; as such, their "Verified Deterministic?" columns are marked as "No" in the table.

Table 7.1: All blocks that were checked for determinism.

Name	Verified Deterministic?
AddressDecompose	Yes
AddressVerify	Yes
AddU32	Yes
BigIntBlock	**No**; counterexample found
FetchBlock	Yes
GlobalsBlock	Yes
GetSign	**With** the `multi-solver`
DecodeBlock	Yes
DestReg	**With** the `multi-solver`; see Subsection 7.4.2
DualReg	**With** the recommendations in Subsection 7.4.1
EcallBigIntBlock	Yes
EcallP2Block	**No**; counterexample found
EcallReadBlock	Yes
EcallWriteBlock	Yes
EcallTerminateBlock	Yes
ExpandU32	**With** determinism annotations; see Subsection 7.4.1
InstAuipcBlock	Yes

InstBranchBlock	Yes
InstEcallBlock	Yes
InstImmBlock	Yes
InstJalBlock	Yes
InstJalrBlock	Yes
InstMretBlock	Yes
InstRegBlock	Yes
InstResumeBlock	Yes
InstSuspendBlock	Yes
IsZero	Yes
MakeTableBlock	Yes
Multiply	**With** determinism annotations; see Subsection 7.4.1
NegU32	**With** an assumption; see Subsection 7.4.3
OneHot	Yes
PageInNodeBlock	Yes
PageInPageBlock	Yes
PageInPartBlock	Yes
PageOutNodeBlock	Yes
PageOutPageBlock	Yes
PageOutPartBlock	Yes
PageUncleBlock	Yes
PhysMemReadBlock	No; depends on parent component
PhysMemWriteBlock	No; depends on parent component
ReadByteBlock	Yes
ReadWordBlock	Yes
SourceReg	Yes
SplitTotal	Yes
TwoHot	Yes
UnitAddSubBlock	Yes
UnitBitBlock	Yes
UnitMulBlock	Yes
UnitDivBlock	**No**; see V-R0M3-VUL-001
UnitLtBlock	Yes
UnitShiftBlock	Yes

7.3.2 Other Functional Correctness Properties

The Veridise security analysts also used Picus to validate functional correctness properties specific to each component when exploitability of potential vulnerabilities was unclear. These properties are described below and in Section 7.4.

AddressVerify isMM correctness

In V-R0M3-VUL-010, it is observed that the `AddressVerify` component computes a value `isMM` that is supposed to be 1 when the current privilege mode is machine mode, or 0 otherwise. This is implemented as the expression `isMM = mode * inv(MODE_MACHINE)` and may not be a bit value

when mode is not equal to 0 or `MODE_MACHINE`. A malicious prover could potentially exploit this fact to bypass the constraint imposed by `AddressVerify`.

To validate the impact of this vulnerability, the security analysts added a post-condition of `mode == 0 || mode == 3` to the Picus file. If this property holds, then valid witnesses may only satisfy the `AddressVerify` constraint when the privilege mode is user mode or machine mode. Picus is able to verify this post-condition, proving that a malicious prover cannot bypass the constraint even if in another privilege mode.

JALR instruction clearing out the low bit

Picus was used to confirm whether the `InstJalrBlock` clears the low bit of the branch target computed for the `JALR` instruction. This is done by adding the following commands to the Picus file extracted from the `InstJalrBlock`:

- Output `t, b`
- Assume `writeRd.data.low == 2t + b`
- Post-condition `b == 0` (i.e., `writeRd.data.low` must be even)

Picus finds a counterexample, which implies that the computed branch target can have an odd value. This was reported as vulnerability V-R0M3-VUL-006.

Decoding instructions at unaligned PCs

Picus was used to confirm whether the `DecodeBlock` allows the PC to be set to an odd address. This validation was performed by adding the following commands to the Picus file extracted from the `DecodeBlock`:

- Output `t, b`
- Assume `pc.low == 2t + b`
- Post-condition `b == 0` (i.e., `pc.low` must be even)

Picus finds a counterexample, which implies that `pc.low` can have an odd value. The consequences of this are described in V-R0M3-VUL-007.

Decoding instructions in protected memory

Picus was used to confirm whether an instruction can be read from kernel memory while running in the user privilege mode. To validate this, the following commands were added to the Picus file extracted from the `DecodeBlock`:

- Assume `mode == 0`: The current privilege mode is user mode.
- Post-condition `load0.wordAddr < 0x30000000`: This checks that the first half of the instruction word is within user memory.
- Post-condition `load1.wordAddr < 0x30000000`: This checks that the second half of the instruction word is within user memory.

Picus finds a counterexample for the second post-condition where `load1.wordAddr` is set to `0x30000000`, leading to V-R0M3-VUL-007.

7.4 Recommendations

After reviewing the Picus annotations present in the circuit source code, the Veridise analysts recommend that the Picus annotations be improved in three key aspects:

1. Bounds Post-conditions
2. Determinism Annotations
3. Argument and Component Invariants

7.4.1 Determinism Annotations

The Veridise analysts recommend updating the Picus determinism annotations as suggested below.

- When a component is extracted to a Picus module, the parameters are translated to inputs, while all member values and intermediate values are translated to outputs. In the circuit code, there are several locations where member values that are conceptually considered to be "inputs" are annotated to be assumed deterministic. This may be unsound when used with Picus `call` expressions, because Picus will *assume* the determinism of such members instead of proving that they are indeed deterministic. The correct way to handle such "input" values is to translate them to Picus module inputs.
- A documentation comment on the `picusArgument` function in `risc0/circuit/rv32im-sys/cxx/compiler/extractor/PicusDirectives.h` states: "Emit a 'DeterministicIf' directive into the IR. This indicates to Picus that if all of the input signals are deterministic, then all of the output signals are also deterministic." The comment therefore suggests that when no inputs are provided, then Picus must prove that the outputs are deterministic.
  However, throughout the code base, there are several locations where `PICUS_ARGUMENTS` with no inputs is abused to mean "assume the given outputs are deterministic". This is also the actual behavior in Zirgen's Picus emitter.
  To avoid confusion, it would be better to introduce a separate `PICUS_ASSUME_DET` macro/helper function which serves this purpose. There are cases where it is useful to prove that a non-output expression is deterministic, but there is no way to do that without writing a `PICUS_ARGUMENT` with non-empty inputs.
- The developers did not check the `EcallBigIntBlock` using Picus and should do so.
- The `BIGINT_STATE_ARGUMENT` annotation is unsound, because it falsely assumes that the determinism of the tuple `(cycle + 1, biPc, mm)` implies that there is a unique tuple `(cycle + cycleCount + 1, biPc + cycleCount, mm)`. However, as shown by V-R0M3-VUL-004, there is no constraint on `cycleCount` that relates it to any of the other parameters in the tuple. If this annotation were to be removed and the `cycleCount` to be properly constrained, then Picus would be able to deduce the determinism of `cycle + cycleCount + 1`.
- The `DualReg` component cannot be proven determinstic by Picus. The reason is that the `sameReg` is marked as deterministic, which is unsound because `rs1Idx` and `rs2Idx` would then be required to be deterministic. However, when `sameReg = 0`, there is more than one pair `(rs1Idx, rs2Idx)`.
  Instead, `rs1Idx` and `rs2Idx` should be marked as inputs, since they will likely be provided by the decoder (and therefore fixed to single values). An additional assumption will then need to be made to ensure that the impossible case of `rs1Idx == rs2Idx, sameReg == 0` is not possible.
  Picus can verify the `DualReg` component with the following additional commands:

```
(assume (=> (= layout_rs1Idx layout_rs2Idx) (> layout_sameReg_inner 0)))
; NOTE: prefer marking rs1Idx as inputs for composibility
(assume-deterministic layout_rs1Idx)
(assume-deterministic layout_rs2Idx)
```

- `ExpandU32` fails to verify as deterministic, because `b0`, `b1`, `b2`, and `b3` are marked as output values, even though they are technically input values. When those values are converted into inputs, then Picus can verify determinism.
- `Multiply` also fails to verify as deterministic due to the `ExpandU32` problem and a similar issue where `signA` and `signB` are marked as outputs. When `signA` and `signB` are marked as inputs, then Picus can verify the determinism of `Multiply`.

7.4.2 Bounds Post-conditions

Several components and helper functions in the circuit make assumptions about the bounds of members/values, such that the soundness of the circuit relies on those assumptions. The Veridise analysts strongly recommend inserting Picus `post-condition` commands in the following components or helper functions to prove that these assumptions indeed hold.

- The `RegU32` component has an implicit assumption that its `low` and `high` members contain U16 values.
- The `cond` function in `risc0/circuit/rv32im-sys/cxx/rv32im/circuit/u32.h` is error-prone because it assumes that the condition parameter `c` is a bit value. If a non-bit value is passed as the condition, then the resulting value will be an arbitrary linear combination of the operands `a, b`.
- The `verify` function of both `SourceReg, DestReg` compute a value `isMM` that is intended to be a bit value that is used with `cond`. However, it is theoretically possible for `isMM` to *not* be a bit when the privilege mode is set to the (unsupported) supervisor mode*. This can be confirmed by Picus using the following input file:

```
(prime-number 2013265921)
(begin-module SourceReg)
(input wordAddr)
(input mode)
(output layout_idx)
(output isMM)
(assert (= isMM (* (* mode (- mode 1)) 1677721601)))
(assert (= (- (+ (+ (* isMM 1073725440) (* (- 1 isMM) 1073725472)) layout_idx)
    wordAddr) 0))
(assert (=> (det mode) (det isMM)))
(post-condition
  (=>
    ; if mode is valid
    (&& (<= 0 mode) (< mode 4))
    ; then isMM should be a bit
    (|| (= isMM 0) (= isMM 1))))
(end-module)


Output:

Analyzing problem: "SourceReg"
```

* A related issue V-R0M3-VUL-010 was raised during the manual code review

```
Verification result for module SourceReg is: invalid postcondition: "((4 <= Var
    (1 := mode)) | ((Var(0 := isMM) = 0) | (Var(0 := isMM) = 1)))"
 Counterexample = {
    "mode": 2,
    "isMM": 1342177281,
    "wordAddr": 402636821,
    "layout_idx": 0,
}
```

- The `SourceReg, DestReg` components each have an `idx` member representing a register number that is assumed to be strictly less than 32. Additionally, for `DestReg`, a large value of `idx` can cause a determinism issue, where multiple large `idx` values can map to the same `wordAddr`. When the bounds are enforced on `idx`, Picus can prove that `DestReg` is deterministic.

7.4.3 Argument and Component Invariants

Picus does not directly support reasoning about arguments. However, it is possible to use Picus assumptions and post-conditions to validate and enforce specific invariants involving arguments.

For example, one invariant of the circuit is that for `DecodeArgument`, the register indices in `rd`, `rs1`, and `rs2` must each be strictly less than 32. When the `DecodeBlock` pushes a `DecodeArgument`, it must be proven that the values satisfy this bound. However, when a component pulls a `DecodeArgument`, it is safe to *assume* that those values satisfy the bound.

The Veridise analysts recommending adding annotations for arguments using a four step process:

1. Identify the invariants of interest for the argument. For example, the `DecodeArgument` has the property that $0 \leq rs1 < 32$.
2. Create a "output" macro that encodes those properties as Picus post-condition commands, and invoke the macro when pushing the argument.
3. Create an "input" macro that encodes those properties as Picus assume commands, and invoke the macro when pulling the argument.
4. Create a "determinism" macro that encodes the functional dependencies between various members of an argument. These correspond to the existing `ARGUMENT` macros already in the circuit source code.

Similarly, components may also have invariants associated with them that are important for overall correctness. The `NegU32` component has two parameters: an input 32-bit integer and `neg`. It assumes that the input consists of two 16-bit limbs, and that `neg` is a bit value (inferred from how `NegU32` is used in `UnitDivBlock`). However, `NegU32` does not enforce these requirements with constraints, so Picus fails to verify determinism for `NegU32`. The counterexample found by Picus is a witness where `in.low == 32768, in.high == 1006665728, neg == 1006632961`, but `outLow == 0, outHigh == 1`. After adding assumptions restricting the ranges of `in.low`, `in.high`, and `neg`, Picus can verify determinism for `NegU32`.

When `NegU32` is used as a subcomponent in another component, such as by `UnitDivBlock`, it is important to check that the assumptions made by `NegU32` are actually satisfied. Enforcement can be implemented by adding Picus post-condition commands in the parent component.

A Appendix

A.1 Intended Behavior: Non-Issues of Note

A.1.1 V-R0M3-APP-VUL-001: Underconstrained BigInt NOP write

Severity	Medium	**Commit**	da8083d
Type	Underconstrained Circuit	**Status**	Intended Behavior
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/circuit/bigint.ipp`		

Description When a BigInt instruction sets a `memOp` to `WRITE`, the `BigIntBlock` does not impose any constraints on the value of its `bytes` register. Instead, the `AccumTop::verify` method is responsible for constraining the `bytes` register so that it must satisfy the constraints related to the `polyOp` that is performed.

However, when the BigInt instruction also has the `polyOp` set to `NOP`, then the `bytes` register is effectively unconstrained. The constraints involved with the `NOP` operation in `AccumTop::verify` do not involve the `bytes` data.

```
  for (size_t i = 0; i < 16; i++) {
    localGoal += cur * top->mux.BigInt.data[0].bytes[i].get();
    // ...
  }
  // ...
  localGoal = localGoal * isBigInt;
  ctx.eqz(localGoal - local.get());
  // ...
#define XEQ(op, lhs, rhs) ctx.eqz(((lhs) - (rhs)) * polyOp.bits[uint32_t(op)].get());
  // PolyOp::NOP
  XEQ(PolyOp::NOP, poly.get(), ValExt<C>(0));
  XEQ(PolyOp::NOP, term.get(), ValExt<C>(1));
  XEQ(PolyOp::NOP, total.get(), ValExt<C>(0));
```

Snippet A.1: Lines of code in `AccumTop::verify` demonstrating the constraints on `NOP`. There is no relationship between the BigInt `bytes` and the `poly, term, total` registers in `AccumTop`.

Impact For BigInt instructions with `memOp = WRITE` and `polyOp = NOP`, a malicious prover can generate a valid witness where the bytes written to the target address may be set to any arbitrary value.

Recommendation To be extra safe, add a constraint to `BigIntBlock::verify` that prevents using a `WRITE` in combination with a `NOP`.

© 2026 Veridise Inc.

Developer Response The developers have indicated that this is intended behavior:

> This is intended to allowed: For polyOp to be NOP requires the BigInt program passed by the user to do so. This could be used for example to prefill a nondeterministic output (although I don't believe any current BigInt programs do this). Regardless, since this is under control of the user, it'd not a soundness issue here.

A.1.2 V-R0M3-APP-VUL-002: EcallTerminateBlock does not validate halt state

Severity	Warning	**Commit**	da8083d
Type	Underconstrained Circuit	**Status**	Intended Behavior
Location(s)	`risc0/circuit/rv32im-sys/cxx/rv32im/[...]/ecall.ipp:82`		

Description When a segment is terminated with using the terminate `ECALL`, the register A0 is expected to store two 16-bit values: a user-specified exit code (in the high limb), and the halt type (in the low limb). Based on the behavior of the `Claim::exit_code` method in `risc0/circuit/rv32im/src/lib.rs`, the halt type is required to either be `TERMINATE` (0) or `PAUSE` (1). However, the `EcallTerminateBlock` does not impose any constraints on the value stored in register A0.

Impact A prover could construct a witness that terminates with an invalid halt code, such that the witness would pass the evaluation check involving the verifier polynomial. Currently, the assumption is that the verifier will make the check outside of the circuit but if at a later date that assumption is invalid, then the circuit should be changed.

Recommendation Consider adding a constraint to `EcallTerminateBlock::verify` that ensures that the halt type is a valid halt type.

Developer Response The developers stated that leaving register A0 unconstrained is intended behavior, in case the set of valid halt types is expanded in the future. Verifier implementations will reject traces with unsupported halt types, so soundness problems arising from this issue are unlikely.

Glossary

M3 arithmetization Multi-multiset matching (abbreviated as M3) is an arithmetization scheme used for the Binius proof system. To learn more, visit https://www.binius.xyz/basics/binius-v0/m3. 1

zero-knowledge circuit A cryptographic construct that allows a prover to demonstrate to a verifier that a certain statement is true, without revealing any specific information about the statement itself. See https://en.wikipedia.org/wiki/Zero-knowledge_proof for more. 1, 5, 34, 44

Zirgen A doamin-specific language developed by RISC Zero for creating zero-knowledge circuits compatible with the RISC Zero proof system. To learn more, visit https://github.com/risc0/zirgen. 1

zkVM A general-purpose zero-knowledge circuit that implements proving the execution of a virtual machine. This enables general purpose programs to prove their execution to outside observers, without the manual constraint writing usually associated with zero-knowledge circuit development . 1

© 2026 Veridise Inc.